

NuJuliet

Radek Folprecht

January 27, 2026

Manual for Nuclear Many-body Physics Julia Solver

NuJuliet is a *Julia* language solver ("package") for many-body nuclear structure calculations. The code is actively developed by R. Folprecht. Parts of the code are inspired by HFB nuclear structure code of F. Knapp & P. Veselý [1].

As of today, the code features these many-body approximations:

- Spherical Hartree-Fock (**HF**) mean-field method,
- Leading-order HF-Many-body Perturbation Theory (**LO HF-MBPT**),
- HF-Tamm-Dancoff Approximation (**HF-TDA**),
- HF(-Renormalized)-Random-Phase Approximation (**HF-(R)RPA**),
- Spherical HF-Bardeen-Cooper-Schrieffer (**HF-BCS**) theory method,
- Spherical Hartree-Fock-Bogoliubov (**HFB**) mean-field method.
- Quasiparticle TDA (**QTDA**) solver - currently work in progress, the solver is functional.

As input the code requires precomputed 2-body NN & 3-body NNN interaction matrix elements in the Linear Harmonic Oscillator basis as produced by NuHamil code [2] in the binary format, moreover beyond mean-field calculations, e.g. TDA, RPA, ..., require previous mean-field solutions to run. Proper formatting of the input is explained in detail below. Example set of matrix elements is included under NuJuliet/IO/NN.bin & NuJuliet/IO/NNN.bin ($\Delta N^2 \text{LO}_{\text{GO}}$ (394), $\hbar\omega = 16$ MeV, $N_{\text{max}} = 3$, $N_{3\text{max}} = 9$).

Software Requirements

To be able to run NuJuliet one has to have installed *Julia* language with version $\geq v1.9$. **Preferably you should use the Long-Term-Support release of Julia v1.10.** More up-to-date versions (v1.11 or the current v1.12) are fine as well, but cause significant performance drops with respect to v1.10 (*changes in compiler handling of dictionary operations ??? ... ~ 10x slower*). To install *Julia* manually or via Juliaup see [3]. **In addition, the solver requires these *Julia* packages:**

LinearAlgebra, BenchmarkTools, Printf, DelimitedFiles, Mmap, CGcoefficient, Hungarian

To install these, you can use the built-in package manager Pkg, e.g. install **CGcoefficient** via

```
$ julia
$ using Pkg
$ Pkg.add("CGcoefficient")
```

Running Scripts

In order to run NuJuliet one needs to prepare a script and run it using *Julia*, e.g.

```
$ cd solver_placement/NuJuliet/  
$ julia Scripts/sample_script_run.jl
```

To ensure proper linking, always call your scripts from `solver_placement/NuJuliet/`. The folder `solver_placement/NuJuliet/IO/` contains calculation outputs as well as inputs for further function calls (*residual interaction export etc.*). Handling of `IO/` is explained further.

Scripts Structure

Next, the structure of scripts which are used to perform the solver calls is explained. Note the ordering of keyword entries used below is not fixed in *Julia*.

Interaction_Parameters

First, every script needs to feature the structure `Interaction_Parameters()` that defines the interaction parameters and provides link to the files with NN & NNN interaction matrix elements in binary format as generated by NuHamil. Example is given below.

```
IntParams = Interaction_Parameters(  
  NN_File = "IO/NN.bin",  
  NNN_File = "IO/NNN.bin",  
  hw = 16.0,  
  Nmax = 3,  
  N2max = 6,  
  N3max = 9,  
  cRes = 1.0,  
  cP2N = 1.0,  
  cP3N = 1.0  
)
```

In the example above the variable `IntParams` is of type `Interaction_Parameters()`, which is a structure bearing the information regarding the internucleon potential matrix elements as well as configuration space properties. The entries highlighted in red are mandatory and always need to be specified for any calculations to run, all the remaining fields, i.e. `cRes`, `cP2B`, ... can be safely omitted, i.e. these are optional and if not specified, they are put to default values. A short overview of entries available in `Interaction_Parameters()` is provided.

- `NN_File` - String containing the path to the NN interaction matrix elements.
- `NNN_File` - String containing the path to the NNN interaction matrix elements.
- `hw` - Float64 value of the oscillator frequency $\hbar\omega$ in units of MeV of the LHO basis used.
- `Nmax` - Int64 value of the maximum of LHO shells $N_a = 2n_a + l_a$ used for the LHO single-particle basis $\{|a\rangle\}_a$ (maximal tested value is $N_{\max} = 16$).
- `N2max` - Int64 value of the maximal value of 2-body LHO shell $N_{ab} = 2(n_a + n_b) + l_a + l_b$ used in the LHO single-particle basis $\{|ab\rangle\}_{a,b}$ (should be set to $N_{2\max} = 2N_{\max}$).

- **N3max** - Int64 value of the maximal value of 3-body LHO shell $N_{abc} = 2(n_a + n_b + n_c) + l_a + l_b + l_c$ used in the LHO single-particle basis $\{|abc\rangle\}_{a,b,c}$ (in principle, the maximal value is due to $N_{3max} = 3N_{max}$, in practice though, one cannot go beyond $N_{3max} = 28$ because NuHamil is not able to generate larger configuration spaces).
- **cRes** - Float64 value which scales the 2-body residual density-dependent interaction. By default, the value is set to 1.0 - no rescaling.
- **cP2B** - Float64 value which scales the 2-body matrix elements entering the pairing channel in BCS & HFB calculation. By default, the value is set to 1.0 - no rescaling.
- **cP3B** - Float64 value which scales the 3-body matrix elements entering the pairing channel in BCS & HFB calculation. By default, the value is set to 1.0 - no rescaling.

Calculation_Parameters

Second vital input is due to structure `Calculation_Parameters` which carries all the essential information concerning the calculation details, such as the target nucleus, the configuration space size, as well as the optional constraints for solver. Once again, a basic example is provided.

```
CalcParams = Calculation_Parameters(
A = 16,
Z = 8,
hw = 16.0,
Nmax = 3,
N2max = 6,
N3max = 9,
CMS = "CMS1+2B",
Path = "A16_Z8_hw16.0_Nmax3_N2max6_N3max9_CMS1+2B",
HF = HF_Parameters(Tol = 1e-7, Imax = 100, BMF = true, HRF = false),
RPA = RPA_Parameters(Ortho = true),
RRPA = RRPA_Parameters(Ortho = true, Tol = 1e-7, Imax = 100,
                        ScV3N = true, ScOBH = true, dOBDM = false),
BCS = BCS_Parameters(ScBCS = true, Tol = 1e-7, Imax = 500, q = 0.05,
                     A0 = 0, Z0 = 0, pD0 = 0.5, nD0 = 0.5,
                     BMF = false, HRF = false),
HFB = HFB_Parameters(Tol = 1e-7, Imax = 150, dLmax = 0.5, BMF = true,
                     HRF = false),
QTDA = QTDA_Parameters(Ortho = true),
)
```

The variable `CalcParams` in the instance above is of type `Calculation_Parameters()`, a structure which carries the specific details of the calculation to be performed in solver calls. As before, **the fields in red are mandatory, and must be specified for solver to run properly**, all the remaining fields, i.e. `CMS`, `Path`, `HF`, ... can be safely omitted, i.e. these are optional and if not specified, they are set to **default values**. All the entries available in `Calculation_Parameters()` structure are listed and explained.

- **A** - Int64 value of the nucleon number of the target nucleus to be calculated.
- **Z** - Int64 value of the proton number of the target nucleus to be calculated.

- **hw** - Float64 value of the oscillator frequency $\hbar\omega$ in units of MeV of the LHO basis used. Should be set to the value $\hbar\omega$ used for the interaction potential.
- **Nmax** - Int64 value of the maximum of LHO shells N_a used for the calculation.
- **N2max** - Int64 value of the maximal value of 2-body LHO shell N_{ab} used for the calculation.
- **N3max** - Int64 value of the maximal value of 3-body LHO shell N_{abc} used for the calculation.
- **CMS** - String which can be set to 3 different values CMS = CMS1+2B, CMS2B, NoCMS. Each of these options defines way the code performs the Centre-of-Mass System (CMS) correction. This is performed through subtraction of the CMS kinetic energy either as the combined 1- + 2-body operator, pure 2-body operator or the subtraction is left out. **Default value is set to CMS = "CMS1+2B".**
- **Path** - String variable which defines the solution IO folder in IO/. **By default set to Path = "A(A)_Z(Z)_hw(hw)_Nmax(Nmax)_N2max(N2max)_N3max(N3max)_CMS1+2B".**
- **HF** - HF_Parameters() type that provides optional parameters for the HF calculation. The HF solver parameters are.
 - **Tol** - Float64 value of the solver precision. **By default set to Tol = 1e-7.**
 - **Imax** - Int64 value of the maximal number of HF iterations. **By default set to Imax = 100.**
 - **BMF** - Bool value enables (true) / disables (false) export of beyond mean-field residual 2-body NN interaction and execution of HF-MBPT calculation. **By default set to BMF = true.**
 - **HRF** - Bool enables (true) / disables (false) export of HF single-particle orbitals & residual 2-body interaction in Human-Readable Format (*inefficient & internally not supported*). **By default set to HRF = false.**
- **RPA** - RPA_Parameters() type that provides optional parameters for the HF-RPA & HF-TDA calculation. The RPA solver parameters are.
 - **Ortho** - Bool, controls - enables (true) / disables (false) orthogonalization of the spurious 1^- Nambu-Goldstone state. **By default set to Ortho = true.**
- **RRPA** - RRPA_Parameters() type that provides optional parameters for the HF-RRPA calculation. The RRPA solver parameters are.
 - **Ortho** - Bool, controls - enables (true) / disables (false) orthogonalization of the spurious 1^- Nambu-Goldstone state. **By default set to Ortho = true.**
 - **Tol** - Float64 value of the solver precision. **By default set to Tol = 1e-7.**
 - **Imax** - Int64 value of the maximal number of HF-RRPA iterations (both for OBDM iteration as well as diagonalization iterations). **By default set to Imax = 150.**
 - **ScV3N** = Bool value that enables (true) / disables (false) fully self-consistent treatment of 2-body density-dependent interactions, i.e. as OBDM is updated, the density-dependent component of interaction is updated as well. **By default set to ScV3B = true.**
 - **ScOBH** = Bool values that controls (true / false) update of 1-body mean-field Hamiltonian throughout iterations. Otherwise the static HF mean-field is used. **By default set to ScOBH = true.**

- dOBDM - Bool value that determines (true / false) whether diagonal approximation for the 1-body density matrix (OBDM) is used. By default set to dOBDM = false.
- BCS - BCS_Parameters() type that provides optional parameters for the BCS calculation. The BCS solver parameters are.
 - Tol - Float64 value of the solver precision. By default set to Tol = 1e-7.
 - Imax - Int64 value of the maximal number of HF-BCS iterations. By default set to Imax = 500.
 - ScBCS - Bool value that enables (true) / disables (false) self-consistent iteration of HF & BCS, i.e. for every converged BCS calculation, the amplitudes U & V are used to produce new densities ρ & κ which are used to determine new HF mean-field single-particle orbitals & energies $\{\epsilon_a\}_a$, which are again used to find new BCS solution, and so on. By default set to ScBCS = false.
 - A0 = Int64 value that specifies the nucleon number A for reference mean-field calculation. By default set to A0 = 0, in which case the solver picks up the nearest double-magic value of A smaller than or equal to the target nucleon number of the open-shell system.
 - Z0 = Int64 value that specifies the proton number Z for reference mean-field calculation. By default set to Z0 = 0, in which case the solver picks up the nearest double-magic value of Z smaller than or equal to the target proton number of the open-shell system.
 - pD0 - Float64 value in units of MeV that determines the initial constant guess on the BCS proton pairing gap $^p\Delta$. By default set to pD0 = 0.5.
 - nD0 - Float64 value in units of MeV that determines the initial constant guess on the BCS neutron pairing gap $^n\Delta$. By default set to nD0 = 0.5.
 - q - Float64 value in units of MeV that controls the quenching of chemical potential λ in BCS iteration, i.e. $\lambda \rightarrow \lambda + q (A - \langle N \rangle)$. In case the BCS iteration does not converge, I suggest to lower the value of q . Note it is same for quenching of both proton & neutron chemical potentials. By default set to q = 0.05.
 - BMF - Bool value enables (true) / disables (false) export of beyond mean-field residual 2-body NN interaction and execution of HF-BCS-MBPT calculation. By default set to BMF = true.
 - HRF - Bool enables (true) / disables (false) export of HF & BCS single-particle orbitals & residual 2-body interaction in Human-Readable Format (*inefficient & internally not supported*). By default set to HRF = false.
- HFB - HFB_Parameters() type that provides optional parameters for the HFB calculation. The parameters in gray should be mostly irrelevant for basic usage. The HFB solver parameters are.
 - Tol - Float64 value of the solver precision. By default set to Tol = 1e-7.
 - Imax - Int64 value of the maximal number of HFB iterations. By default set to Imax = 150.
 - dLmax - Float64 value in units of MeV that fixes the maximal allowed change of chemical potential in HFB iterations. In case the solver struggles to satisfy the particle number constraint, lower the value of dLmax. Applies to both proton & neutron chemical potentials. By default set to dLmax = 0.5.
 - pL0 - Float64 initial value of proton chemical potential in units of MeV. By default set to pL0 = -3.0.

- nL0 - Float64 initial value of neutron chemical potential in units of MeV. By default set to nL0 = -3.0.
 - pa0 - Float64 value of diffuseness of Woods-Saxon distribution used to generate initial proton densities ${}^p\rho$ & ${}^p\kappa$. By default set to pa0 = 0.25.
 - na0 - Float64 value of diffuseness of Woods-Saxon distribution used to generate initial neutron densities ${}^n\rho$ & ${}^n\kappa$. By default set to na0 = 0.25.
 - Broy_m - Int64 value defining the number of history vectors for the modified Broyden's algorithm used for density updates in HFB iteration. By default set to Broy_m = 8.
 - Broy_Qmax - Int64 value defining the maximal number of update damping (by half) when performing the Broyden's update. By default set to Broy_Qmax = 3.
 - Broy_Amax - Float64 value defining the initial magnitude of density mixing in Broyden's update. Each damping step decreases it by half. By default set to Broy_Amax = 0.9.
 - Broy_Bmax - Float64 value defining the maximal magnitude of step update due to history vectors with respect to density mixing in Broyden's update. By default set to Broy_Bmax = 0.3.
 - Broy_Tmax - Float64 value defining the maximal magnitude of trial step in Broyden's update with respect to previous solution (needs to be < 1 to ensure convergence). By default set to Broy_Tmax = 0.9.
 - BMF - Bool value enables (true) / disables (false) export of beyond mean-field residual 2-body NN interaction and execution of HFB-MBPT calculation. By default BMF = true.
 - HRF - Bool enables (true) / disables (false) export of HFB single-particle orbitals & residual 2-body interaction in Human-Readable Format (inefficient & internally not supported). By default set to HRF = false.
- QTDA - QTDA_Parameters() type that provides optional parameters for the QTDA calculation. The QTDA solver parameters are.
 - Ortho - Bool, controls - enables (true) / disables (false) orthogonalization of the spurious 1^- Nambu-Goldstone state. By default set to Ortho = true.

Solver Calls

With configuration & calculations parameters defined, one can proceed to perform solver calls. The individual solvers are called using corresponding functions with Interaction_Parameters & Calculation_Parameters as function call inputs, e.g. the HF call followed by the HF-RPA call is executed as follows:

```
HF(Parameters(IntParams,CalcParams))
HF_RPA(Parameters(IntParams,CalcParams))
```

where the variables IntParams & CalcParams are just the ones defined earlier. Further, all the solver call functions available are listed.

- HF() - The HF mean-field solver.
- HF_RPA() - The standard HF-RPA & HF-TDA 1-phonon calculation.
- HF_RRPA() - The sophisticated HF-RRPA solver.

- BCS() - Standard HF based spherical BCS calculation for even-even open-shell nuclei.
- HFB() - Standard spherical HFB solver for even-even open-shell nuclei.
- QTDA() - *Work in progress* spherical QTDA 1-phonon calculation based on reference quasiparticle mean-field (HFB/BCS) solution.

Parallelization of NuJuliet

By default the solver supports basic parallelization, the multi-threading in particular. To run given script at multiple threads, make sure you have access to needed threads & export the desired number of threads before calling the script, e.g.

```
$ export JULIA_NUM_THREADS=24
$ cd solver_placement/NuJuliet/
$ julia Scripts/sample_script_run.jl
```

Further parallelization techniques could involve parallelization of scripts that call the solvers (sample_script_run.jl), these are not included in example scripts.

Note that running mean-field solvers with 12-24 CPUs is just enough, for RRPA it makes sense to go up to 48. In case of TDA & RPA you can safely stick to 1 CPU.

References

- [1] F. Knapp; P. Veselý. HF(B) nuclear structure many-body solver, https://github.com/knappf/HF_3b_no2b.
- [2] T. Miyagi. NuHamil : A numerical code to generate nuclear two- and three-body matrix elements from chiral effective field theory. *European Physical Journal A*, 59:150, 2023.
- [3] Julia language, <https://julialang.org/>.